

cache performance metrics

hit latency: time it takes to look in the cache and retrieve the requested data when get **cache hit**

- e.g. often **3 or 4 clock cycles** for **primary caches**

miss rate: fraction of memory references not found in the cache

- miss rates of **under 5%** are common for **primary caches**, **under 2%** for **primary/secondary cache combination**

miss penalty: time required for a block to be retrieved from the next level

- note that typically have very large miss penalty if have to go to **main memory** ... could be **multiple hundreds of clock cycles!**

Processor cache organization

will look first at the simplest organization, called **direct mapped**, then will look at a closely related organization called **set associative**

direct-mapped cache

a big array (or table); at **each position (row)**, store the contents of a **single memory block**, as well as:

- some **status bits**
- a **tag field** that specifies **which memory block** has been stored there

when a copy of a memory block is put in the cache, what position is used?

- when memory block number i is copied into a direct-mapped cache with M positions, it is put in position $i \bmod M$
- since each position has space for just a single block, the new block will **replace whatever memory block was there before**

use of tag field

any block that would be put in position j ($0 \leq j < M$) in the cache has a block number that gives a **remainder** of j when divided by M

... and so the **tag field** at each position j need only store the **integer part** when the block number is divided by M

direct-mapped cache example

to make example simpler, consider a cache much smaller than those used in practice, with just $M = 16$ positions

assume a block size B of **32 bytes** (or 8 words), and suppose do reads (e.g. “lw”) on the following memory addresses: **80, 1040, 948, 1864, 836**

what memory blocks get put in the cache, and where, and what are the final cache contents???

first step: which memory blocks are being read?

address	memory block number
80	2 (= integer part when divide the address (80) by the block size (32))
1040	32 (= integer part of 1040/32)
948	29 (= integer part of 948/32)
1864	58 (= integer part of 1864/32)
836	26 (= integer part of 836/32)

now determine cache positions and tag fields

memory block #	cache position	tag field value
2	2 (= 2 mod 16)	0 (= int. part of 2/16)
32	0 (= 32 mod 16)	2 (= int. part of 32/16)
29	13 (= 29 mod 16)	1 (= int. part of 29/16)
58	10 (= 58 mod 16)	3 (= int. part of 58/16)
26	10 (= 26 mod 16)	1 (= int. part of 26/16)

final cache contents

memory blocks 2, 32, 29, 26 (not memory block 58)

suppose now we read the word with memory address 1028, will we get a cache hit or a cache miss?

using fields from memory address to determine
memory block number, cache position, and tag field

simple if M , B chosen as powers of 2: $M = 2^m$, $B = 2^b$
rightmost b bits of memory address give the relative
position of the byte with that address within its
memory block

remaining $32 - b$ bits give the **memory block
number**

rightmost m bits of the memory block number give
the **cache position** that the block would be put in
remaining $(32 - b) - m$ bits give the **tag field value**

example

$M = 2^4$, $B = 2^5$, memory address 1864

in binary, 1864 is 0 0 ... 0 1 1 1 0 1 0 0 1 0 0 0

what is the **memory block number**?

what is the **cache position**?

what is the **tag field value**?

advantages/disadvantages of direct-mapped cache

advantage: fast access

disadvantage: no choice of which block to replace
(can lead to bad performance if “unlucky”!)

set-associative cache

in each row (termed a “set”) have storage spaces
for multiple memory blocks

if M sets, storage spaces for E memory blocks per
set, can store $M \times E$ memory blocks in total

- $E = 1$ gives direct-mapped cache
- $M = 1$ gives “fully associative” cache
- common values for E range from 2 to 32

when a copy of a memory block is put in the cache,
where can it be put?

- memory block number i can be put in any of
the storage spaces in set $i \bmod M$